Problem Set 1 — Report

Problem 1: Circular Hough Transform Implementation

The goal was to implement the Standard Circular Hough Transform (CHT) from scratch to detect circles in an image. Unlike the linear Hough transform which uses a 2D accumulator (ρ, θ), the circular transform requires a 3D parameter space *(a,b,r)*, where *(a, b)* represents the circle center and *r* is the radius.

The equation of a circle is:

$$(x - a)^2 + (y - b)^2 = r^2$$

Algorithm Steps:

1. **Preprocessing:** The input image was converted to grayscale to reduce dimensionality.
2. **Edge Detection:** A Canny edge detector (`cv2.Canny`) was applied. This is a crucial step as the Hough transform relies entirely on edge pixels to "vote" for potential shapes.
3. **Accumulator Initialization:** A 3D accumulator array `H` of size $(Height\ x\ Width\ x\ RadiusRange)$ was initialized to zeros.
4. **Voting Process:**
    - For every edge pixel (x, y)found in the Canny output:
    - We iterate through the desired radius range (`min_r` to `max_r`).
    - For each radius *r*, we calculate the locus of points that could be the center of a circle passing through (x, y). This is calculated using polar coordinates:

      $a = x - r \cdot \cos(\theta)$

      $b = y - r \cdot \sin(\theta)$

    - We iterate $\theta \; from \; 0^{\circ} \; to \; 360^{\circ}$. The calculated coordinates *(a, b, r)* in the accumulator are incremented by 1.
5. **Peak Detection:** We threshold the accumulator array using a value *t*. Locations with votes > *t* are considered potential circle centers. These peaks are sorted by vote strength, and the top *s* circles are returned.

Implementation Details

To ensure the Python implementation ran in a reasonable timeframe, several optimizations were used:

- **Trigonometric Lookup:** `sin` and `cos` values were pre-calculated to avoid repeated function calls inside loops.
- **Vectorization:** Instead of looping through every edge pixel individually in Python (which is slow), I used numpy to process the voting geometry for a given radius across the entire image matrix where possible.

Experimental Results

Edge Detection

The Canny edge detector was applied with thresholds `100` and `200`.

The edge detector captured the boundaries of the coins. However, it also captured significant internal texture details like faces on the coins and text.

Hough Transform Results

The algorithm was run with the following parameters:

- `min_radius = 20`
- `max_radius = 100`
- `threshold (t) = 110`

The custom implementation identified the four largest coins in the image.

1. **Accuracy:** The radii of the detected coins appear visually correct, hugging the coin boundaries almost right.
2. **Missed Detections:** The algorithm missed one in the middle left.
    - *Reason might be such that it didnt receive enough 110 votes*
3. **Multiple Detections (Clustering):** In the bottom-center coin, we observe two red center dots close together.
    - *Reason might be* because of the implementation of simple peak finding method. If a pixel *(x, y)* has 115 votes and its neighbor *(x+1, y)* has 114 votes, both are above the threshold and both are drawn.

Comparison with OpenCV

The OpenCV `HoughCircles` function was run for comparison.

Comparison:

- **Method Difference:** OpenCV uses the Hough Gradient Method. Instead of voting for every angle 0..360, it calculates the gradient direction of the edge pixel and only votes along that line. This is significantly faster and uses less memory.
- **Performance:** OpenCV detected all coins. The circles are cleaner, with exactly one circle per coin. This highlights the effectiveness of the gradient method.
- **False Positives:** My custom implementation was stricter (due to the high threshold of 110), whereas OpenCV was more sensitive.

Problem 2: Live Video Demonstration

Applying the custom Hough Transform from Problem 1 to live video was computationally heavy. A 640 x 480 video frame contains over 300,000 pixels. Calculating 360 degrees of voting for every edge pixel resulted in millions of operations per frame, leading to a frame rate of < 0.1 FPS.

Optimization Strategy

To achieve near real-time performance (`ps2_2.py`), the following modifications were made:

1. **Downscaling:** The video frame is resized to a width of *320 pixels* before processing. This reduces the number of pixels to process by factor of ~4.
2. **Reduced Angular Resolution:** Instead of voting for every 1°, the algorithm votes every 6°. This reduces the computational load of the voting loop by a factor of 60 while maintaining enough accuracy for visual detection.
3. **Dynamic Edge Detection (Auto-Canny):** In a video feed, lighting changes constantly. Fixed thresholds (e.g., 100/200) often result in either black images or too much noise. An `auto_canny` function was implemented that sets thresholds based on the median intensity of the current frame:

$$Lower = \max(0, (1.0 - \sigma) \cdot median)$$

$$Upper = \min(255, (1.0 + \sigma) \cdot median)$$

The video demonstration highlights the trade-off between accuracy and speed in Computer Vision as we needed to sacrifice in custom func. While the "Fast" Hough Transform is less precise than the offline version (which may result in slightly jittery circles), though it allows for interactive detection rates necessary for real-time applications.